ডেডলক

কফম্যানের চার শর্ত · RAG চক্র · ব্যাংকার অ্যালগরিদম · ১১ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = যে এক চিত্র থেকেই প্রশ্ন আঁকা হয়

➜ সিলেবাসের যে অংশ

অপারেটিং সিস্টেমের সিলেবাসে আছে "*Deadlock: resource allocation and deadlock, deadlock detection, prevention and recovery*", সঙ্গে ব্যাংকার অ্যালগরিদমে এভয়েড্যান্স। শিডিউলিং ও IPC প্রসেস লেসনে আছে।

হাই-ইল্ড তালিকা

① কফম্যান × ৪: *M, H, N, C*
② ক্লেইম *R→P* · রিকোয়েস্ট *P→R*
③ চক্র মানেই ডেডলক কেবল ১ ইনস্ট্যান্সে
④ *Need = Max − Allocation*
⑤ *Work = Available; Work += Alloc*
⑥ সেফ = প্রতিটি *Finish[i] true*
⑦ অসুরক্ষিত ≠ ডেডলক
⑧ সার্কুলার ওয়েট ভাঙো: অর্ডারিং *a priori*
⑨ পুনরুদ্ধার: টারমিনেট বা প্রিম্পশন
⑩ এজিং ঠিকায় স্টারভেশন, ডেডলক নয়

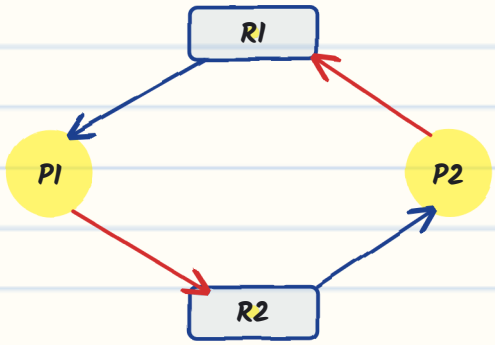
সংকেত হলুদ = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

➜ সূচিপত্র

✓ ডেডলক কী, চিরায়ত ২ × ২ ... পৃ. ২
✓ কফম্যানের চার শর্ত ... পৃ. ৩
✓ রিসোর্সের প্রকার ও RAG ... পৃ. ৪
✓ চক্র মানেই কি ডেডলক? ... পৃ. ৫
✓ প্রতিরোধ: একটি শর্ত ভাঙো ... পৃ. ৬
✓ এভয়েড্যান্স ও নিরাপদ অবস্থা ... পৃ. ৭
✓ ব্যাংকার অ্যালগরিদম ... পৃ. ৮
✓ সনাক্তকরণ ও পুনরুদ্ধার ... পৃ. ৯
✓ রাতের পাতা পৃ. ১০ · অনুশীলন পৃ. ১১

১ ডেডলক কী? সংজ্ঞা ★

সংজ্ঞা এমন একটি প্রসেসের সেট (set) যেখানে **প্রতিটি** সদস্য ব্লক হয়ে আছে এবং এমন কোনো ঘটনার অপেক্ষায় আছে যা কেবল একই সেটের আরেকটি ব্লক করা সদস্যই ঘটাতে পারে। কেউ এগোতে পারে না, বাইরের কেউ সাহায্য করতে পারে না, এবং **পরিস্থিতি নিজে থেকে কখনো মুক্তি পায় না**। একটি প্রসেস কেবল ব্যবহৃত ডিভাইসের জন্য অপেক্ষা করলে তা **ডেডলক নয়** — সেটি সাধারণ ব্লকিং।



নীল = ক্লেইম · লাল = রিকোয়েস্ট

চিরায়ত দুই প্রসেস, দুই রিসোর্সের উদাহরণ। একটি প্রিন্টার (R1) ও একটি ডিস্ক ড্রাইভ (R2), প্রতিটির একটি করে ইনস্ট্যান্স, ব্যবহার এককভাবে।

১. P1 পুরো প্রিন্ট কাজের জন্য R1 প্রিন্টারটি নেয়।
২. P2 ডিস্কের কাজ শেষ করে R1 চায়; সে ব্লক হয়।
৩. P1-এর এখন স্পুল ফাইলের জন্য R2 দরকার; R2 খালি, তাই P1 সেটি নেয়।
৪. P1 এখন **প্রিন্ট** করতে চায় — প্রিন্টারটি ধরে আছে P2, যে P1-এর জন্য অপেক্ষায়। **দুজনেই সারাকাল অবরোধে।**

R1 ও R2-এর **একটি করে ইনস্ট্যান্স**: P1 ধরে R1 আর চায় R2; P2 ধরে R2 আর চায় R1

পরিভাষা	অর্থ	এখানে উদাহরণ
অনুরোধ / রিকোয়েস্ট	প্রসেস এমন ইনস্ট্যান্স চায় যা তার নেই	P1 R1-এর জন্য অপেক্ষা করছে
ধরে আছে / বরাদ্দ	ইনস্ট্যান্সটি তার কাছে আছে, ছাড়েনি	P2 ধরে আছে R2
ডেডলক সেট	পরস্পরকে আটকে রাখা সব প্রসেসের সম্পূর্ণ সেট	{P1, P2}

মনে রাখার কৌশল "ডেডলকের জন্য **অন্তত দুজন** লাগে, আর লাগে **চারটাই**।" একটি প্রসেস হয় স্টার্ভ বা ব্লক হয়; ডেডলকে সবসময় থাকে একটি **সেট** আর একটি **চক্র**। চারটি কফম্যান শর্ত একসাথে থাকতে হয় — **একটিও** ভাঙলে ডেডলক হতে পারে না।

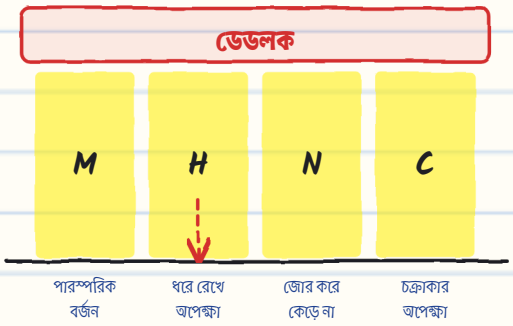
পরীক্ষার ফাঁদ (i) ব্যবহৃত ডিভাইসের জন্য অপেক্ষা করা **ডেডলক নয়**। (ii) **চক্র সবসময় ডেডলক নয়** — পাতা ৫। (iii) ডেডলক ≠ স্টারভেশন: স্টারভেশনে সিস্টেম চলতে থাকে, একটি প্রসেস বঞ্চিত হয়; ডেডলকে **কিছুই** এগোয় না।

সমাধানকৃত উদাহরণ ✍ ছবিটি কথায় পড়ো R1 = একটি প্রিন্টার, R2 = একটি ডিস্ক ড্রাইভ। **(ক)** শুধু P1 → R1: ডেডলক? **না** — P1 কেবল অপেক্ষায়, R1 কেউ ধরে রাখেনি। **(খ)** শুধু P1 → R1 ও P2 → R2: **না** — দুটো ইনস্ট্যান্সই খালি, তাই একজন পেয়ে যাবে, অন্যজন নিরাপদে অপেক্ষা করবে। **(গ)** R1 → P1, P1 → R2, R2 → P2, P2 → R1? **হ্যাঁ** — একটি বদ্ধ শৃঙ্খল, যেখানে প্রত্যেক ধারক পরেরটিকে চায়; এই শৃঙ্খলই **ডেডলক**। **(ঘ)** R2 → P3 যোগ করলে, কিন্তু P3-এর কোনো রিকোয়েস্ট না থাকলে? এখনও **হ্যাঁ** — {P1, P2} চক্রটি নিজে থেকেই দাঁড়িয়ে।

বাস্তবে কোথায় মিলে দুই ডেটাবেস ট্রানজ্যাকশন বিপরীত ক্রমে অ্যাকাউন্ট A তারপর B আপডেট করছে; একটি থ্রেড সকেট পড়ায় ব্লক হওয়ার আগেই মিউটেক্স ধরে রাখছে। শিক্ষা সবসময় এক: **যে ক্রমে জিনিস নেওয়া হয়, সেটি ঠিক করে নিন**।

২ কফম্যানের চারটি প্রয়োজনীয় শর্ত সবচেয়ে বেশি আসে ★

সংজ্ঞা এডওয়ার্ড কফম্যান (*Edward Coffman, ১৯৭১*): ডেডলক সম্ভব হয় তখনই এবং কেবল তখনই, যখন এই চারটি **একই সাথে** থাকে। এগুলি **প্রয়োজনীয় শর্ত**, সংজ্ঞা নয় — একটিও একা ডেডলক তৈরি করে না, আর **একটিও** অসম্ভব করলে ডেডলক অসম্ভব হয়ে যায়।



চারটি স্তম্ভই দাঁড়ালে **ছাদ** পড়ে — একটিও সরালে কিছুই আটকায় না

শর্ত	ঠিক কী বলে	২ × ২ উদাহরণে
M মিউচুয়াল এক্সক্লুশন	**অন্তত একটি** রিসোর্স শেয়ার করা যায় না: একসাথে একটি প্রসেস	একটি প্রিন্টার, একটি ডিস্ক ড্রাইভ
H হোল্ড অ্যান্ড ওয়েট	প্রসেস **অন্তত একটি** রিসোর্স ধরে রেখে **অপেক্ষা** করছে	*P1* ধরে *R1* আর *R2*-এর জন্য অপেক্ষায়
N নো প্রিম্পশন	ধারক নিজে না ছাড়লে রিসোর্স ফেরত আসে না	কেউ নিজে ছাড়ছে না
C সার্কুলার ওয়েট	একটি বদ্ধ শৃঙ্খল: *P0* চায় *P1*-এর ধরা জিনিস, ..., *Pn* চায় *P0*-এর	*P1* চায় *P2*-কে, *P2* চায় *P1*-কে

মনে রাখার কৌশল *MHNC* — "মিউচুয়াল, হোল্ড-অ্যান্ড-ওয়েট, নো-প্রিম্পশন, সার্কুলার"। ভাবো একটা ঘরের **চারটি দরজা**: চারটিই বন্ধ হলেই আপনি আটকে যান। *MCQ*-তে "কোনটি **শর্ত নয়**?" — উত্তর হবে **এজিং (*aging*)**, যা স্টারভেশন ঠেকায়।

পরীক্ষার ফাঁদ *(i)* **অন্তত একটি** রিসোর্স অবাগত্যোগ্য হলেই চলে। *(ii)* সব রিসোর্স শেয়ারযোগ্য হলে ডেডলক হবে না। *(iii)* "সবচেয়ে বাস্তবসম্মত প্রতিরোধ" = টোটাল অর্ডারিং দিয়ে **সার্কুলার ওয়েট** ভাঙা। *(iv)* শর্তগুলো **প্রয়োজনীয়**, পর্যাপ্ত নয়।

সমাধানকৃত উদাহরণ ✍ এখানে কোন শর্তগুলো ভাঙা যায়? একটি ওয়ার্ড প্রসেসার ছাপার সময় প্রিন্ট-কিউ লক, বানান অভিধান ও একটি টেম্পোরারি ফাইল ধরে রাখে; আগে অভিধানটি ডিস্কে লেখে। এর ক্লিপবোর্ড সব অ্যাপ শেয়ার করতে পারে। **(ক)** মিউচুয়াল এক্সক্লুশন — প্রিন্ট কিউ বা লেখার জন্য খোলা টেম্পোরারি ফাইলে এটি **ভাঙা যায় না**; অভিধানের ক্ষেত্রে *read-only* শেয়ার ফাইল বানিয়ে **ভাঙা যায়**। **(খ)** হোল্ড অ্যান্ড ওয়েট — ভাঙা যায়: **প্রিন্ট কিউ নেওয়ার আগে অভিধান ছেড়ে দিন**, অথবা শুরুতেই সব চেয়ে নিন। **(গ)** নো প্রিম্পশন — অভিধানের জন্য ভাঙা যায় (পেজ আউট করে পরে ফেরত আনা যায়), কিন্তু কাগজে ছাপা হয়ে যাওয়া পাতার জন্য **নয়**। **(ঘ)** সার্কুলার ওয়েট — লকের অর্ডারিং দিয়ে ভাঙা যায়: **ক্লিপবোর্ড < অভিধান < টেম্প ফাইল < প্রিন্ট কিউ**।

চার কৌশল এক লাইনে

প্রতিরোধ = আগেই একটি শর্ত অসম্ভব করা
সনাক্তকরণ = তৈরি হওয়ার পরে খোঁজা
এভয়েড্যান্স = প্রতিটি অবস্থা নিরাপদ রাখা
পুনরুদ্ধার = পেয়ে ভেঙে দেওয়া

প্রচলিত ভুল ধারণা মেমোরি ফুরিয়ে গেলে বা ডিস্ক পূর্ণ হলে ডেডলক হয় না — ওগুলো ব্লকিং বা অগ্রগতিহীনতা, ডেডলক নয়। ডেডলকে একটি **সেট** পরস্পরের জন্য অপেক্ষা করে এবং **প্রত্যেকটি** সদস্য নিজের জন্যই আটকা।

৩ রিসোর্সের প্রকার ও রিসোর্স এলোকেশন গ্রাফ RAG ★

সংজ্ঞা রিসোর্স এলোকেশন গ্রাফ (RAG) একটি দিকনির্দেশিত দ্বিপার্থ গ্রাফ। **প্রসেস নোড বৃত্ত, রিসোর্স নোড বর্গাকার**, এবং প্রতিটি ইনস্ট্যান্সের জন্য বর্গের ভেতরে **একটি বিন্দু** (বিন্দু না থাকলে কোনোটি খালি নয়)। প্রতিটি এজ একটি প্রসেস ও একটি রিসোর্সকে যুক্ত করে, কখনো দুই প্রসেস বা দুই রিসোর্সকে নয়।

এজ	দিক	অর্থ
ক্লেইম (অ্যালোকেশন) এজ	রিসোর্স → প্রসেস	ওই ইনস্ট্যান্সটি প্রসেসকে **বরাদ্দ** করা — সে ধরে রাখছে, ছাড়েনি
রিকোয়েস্ট এজ	প্রসেস → রিসোর্স	প্রসেস ওই ধরনের একটি ইনস্ট্যান্স চেয়েছে এবং **অপেক্ষা** করছে

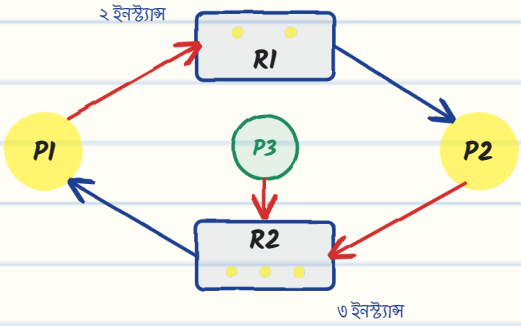
শ্রেণিবিভাগ	অর্থ ও উদাহরণ
রিইউজেবল (ফেরতযোগ্য)	ব্যবহার করে ছেড়ে দিলে আবার পাওয়া যায় — **কখনো ফুরিয়ে যায় না**: CPU, মেমোরি, প্রিন্টার, ডিস্ক, মিউটেক্স
কনজিউমেবল	**ব্যবহারে শেষ** হয়ে যায়, একবার গেলে আর ফেরে না: ছাপা পাতা, স্পুল কাজ, সারির বার্তা
শেয়ারযোগ্য	একই সাথে অনেক প্রসেস ব্যবহার করতে পারে: read-only ফাইল, কোড, শেয়ার read-only মেমোরি
একচেটা (অবাগত্যোগ্য)	একসাথে একটি প্রসেস: প্রিন্টার, টেপ ড্রাইভ, লেখার জন্য খোলা ফাইল, লক

মনে রাখার কৌশল "**R**ইউজেবল মানে **R**িটার্ন (ফেরত); **C**নজিউমেবল মানে **C**নজুম (ব্যবহারে শেষ)।" কনজিউমেবল রিসোর্স কখনো প্রিম্পট হয় না — অর্ধেক ছাপা পাতা অছাপা করা যায় না।

পরীক্ষার ফাঁদ তির দিকই সবচেয়ে বেশি পরীক্ষিত বিষয়।

R → P হলো ক্লেইম এজ (বরাদ্দ); P → R হলো রিকোয়েস্ট এজ (অপেক্ষা)

। যে প্রসেড র চাওয়া **সব** ইনস্ট্যান্সই ধরে আছে, তার কোনো বহির্গত রিকোয়েস্ট এজ নেই — তাই সে **ডেডলকে নেই**।



নীল = ক্লেইম (বরাদ্দ), **লাল = রিকোয়েস্ট** (অপেক্ষা): P1 → R1 → P2 → R2 → P1 চক্রটি আছে, তবু P3 চলতে পারে — R2-এ একটি অতিরিক্ত ইনস্ট্যান্স আছে, তাই এটি **ডেডলক নয়**

৪ চক্র মানেই কি ডেডলক? সবচেয়ে জটিল পার্থক্য ★

এক বাক্যে নিয়ম ডেডলক হলে RAG-তে **অবশ্যই** চক্র তৈরি হয়, তাই চক্র প্রয়োজনীয়। কিন্তু প্রতিটি রিসোর্স ধরনের একটি করে ইনস্ট্যান্স থাকলেই তা পর্যাপ্ত (necessary + sufficient)। একাধিক ইনস্ট্যান্স থাকলে চক্র প্রয়োজনীয় **কিন্তু পর্যাপ্ত নয়**: চক্রের বাইরের কোনো প্রসেস শেষ হয়ে একটি ইনস্ট্যান্স ফেরত দিতে পারে।

পরিস্থিতি	চক্র ⇒ ডেডলক?
প্রতিটি ধরনে ১ **ইনস্ট্যান্স**	হ্যাঁ — সবসময়; শুধু চক্র খুঁজুন
যেকোনো ধরনে > ১ **ইনস্ট্যান্স**	না — হয়তো; রিডিউশন চালান

সমাধানকৃত উদাহরণ ✍ পাতা ৪-এর গ্রাফ রিডিউস করো

R1 = ২ ইনস্ট্যান্স, R2 = ৩ ইনস্ট্যান্স। P1 ধরে ১ × R1, চায় R2; P2 ধরে ২ × R2, চায় R1; P3 কিছু ধরে নেই, চায় R2। **খালি** R1 = 2 − 1 = ১; খালি R2 = 3 − 2 = ১।

১. P3-এর R2 রিকোয়েস্ট মেটে (1 ≥ 1) → **P3 শেষ** হয়, আবার ফেরত দেয়।

২. P1-এর R2 রিকোয়েস্ট এখন মেটে। P1 শেষ হয়ে তার R1 ফেরত দেয় → খালি R1 = 0 + 1 = ১, খালি R2 = 0।

৩. P2-এর R1 রিকোয়েস্ট মেটে (1 ≥ 1)। P2 শেষ হয়।

গ্রাফটি সম্পূর্ণ খুলে যায় ⇒ চক্র আছে কিন্তু ডেডলক নেই।

রিডিউশন পরীক্ষা, ছক্রে

১. প্রতি ধরনের খালি ইনস্ট্যান্স গোনো: **free[j] = total[j] − allocated[j]**।

২. বারবার: এমন প্রসেস P_i খুঁজো যার রিকোয়েস্ট খালি সংখ্যা থেকে মেটানো যায়। ধরো P_i শেষ হয়ে **যা ধরে আছে সব ফেরত দিল**: free[j] += Allocation[i][j]; P_i ফিনিশ বলে চিহ্নিত করো।

৩. সব প্রসেস চিহ্নিত হলে → ডেডলক নেই। কোনো প্রসেস অচিহ্নিত থেকে অপেক্ষায় থাকলে → ডেডলক।

পরীক্ষার ফাঁদ শর্ত ছাড়া কখনো "চক্র = ডেডলক" লিখবেন না। **ডেডলক ⇒ চক্র** সবসময়; **চক্র ⇒ ডেডলক** কেবল এক-ইনস্ট্যান্সের ক্ষেত্রে। এটি অত্যন্ত হাই-ইল্ড: এক-ইনস্ট্যান্স গ্রাফে সঠিক অপশন "ডেডলক **নিশ্চিতভাবে** আছে", বহু-ইনস্ট্যান্সে "ডেডলক **থাকতে** পারে"।

মনে রাখার কৌশল **"এক বিন্দু, এক উত্তর।"** প্রতিটি বর্গে একটিই বিন্দু থাকলে চক্রই প্রমাণ। দুই বা তার বেশি বিন্দু থাকলে কেউ না কারও রিকোয়েস্ট মেটাতে পারে — তাই গ্রাফ **খুলে** দেখতে হবে।

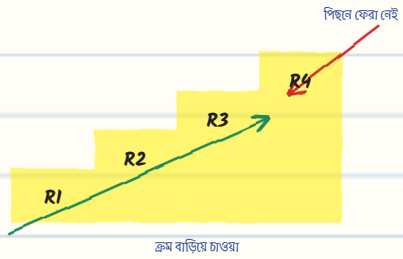
সমাধানকৃত উদাহরণ ✍ R1 = ১ ইনস্ট্যান্স হলে R1 = ১ ইনস্ট্যান্স (P1 ধরে আছে), R2 = ৩ ইনস্ট্যান্স (P2 ধরে ২)। খালি R1 = ১ − ১ = **0**, খালি R2 = ৩ − ২ = ১। P3 শেষ হয়, তারপর দুটো সংখ্যাই ০; P1 বা P2 কেউ এগোতে পারে না। কিছুই খোলে না → P1 → R1 → P2 → R2 → P1 চক্রটি সত্যিকারের ডেডলক।

আইটি লিংক লক-অর্ডারিং বিশ্লেষক চলমান JVM-এর লক গ্রাফের ওপর ঠিক এই রিডিউশনই চালায়। যে গ্রাফ খোলে না, তাতে অন্তত একটি ডেডলক করা থ্রেড পাওয়া যায়।

৫ প্রতিরোধ: চার শর্তের একটি ভাঙা লিখিত পরীক্ষার প্রিয় ★

সংজ্ঞা **প্রতিরোধ (Prevention)** এমন বিধান আরোপ করা যা কোনো প্রসেস চালু হওয়ার আগেই কফম্যানের **অন্তত একটি** শর্তকে স্থায়ীভাবে অসম্ভব রাখে, আগেই। কিছু শনাক্ত হয় না, কিছু পুনরুদ্ধার হয় না — ডেডলক কেবল তৈরিই হতে পারে না।

যে শর্ত অসম্ভব হয়	বিধান	যে দাম
মিউচুয়াল এক্সক্লুশন	রিসোর্সকে **শেয়ারযোগ্য** করো: *read-only* ফাইল, শেয়ার মেমোরি, একটি **স্পুলার**	প্রিন্টার ও লেখার জন্য খোলা ফাইল স্বভাবতই একচেটা → **সবসময় সম্ভব নয়**
হোল্ড অ্যান্ড ওয়েট	**সব একসাথে** চাও, অথবা নতুন চাওয়ার আগে **ধরা সব ছেড়ে দাও**	ব্যবহার কম; প্রতিবার হেরে যাওয়া → স্টারভেশন
নো প্রিম্পশন	অনুরোধ না মিললে প্রসেস যা ধরে আছে **কেড়ে নাও**	*CPU*, **মেমোরি, ফাইলে** কাজ করে; **প্রিন্টারে অসম্ভব**
সার্কুলার ওয়েট ★	সব ধরনে একটি **টোটাল অর্ডার**; রিকোয়েস্ট অবশ্যই **ক্রমানুসারে উচ্চতর** নম্বরের হতে হবে	ভুল ক্রমের অনুরোধে নতুন করে শুরু; কিছুই নষ্ট হয় না, রোলব্যাক লাগে না



প্রতি ধরনকে আগেই *(a priori)* একটি নম্বর দেওয়া হয়; কেবল বর্ধিত ক্রমে চাওয়া যায়

সমাধানকৃত উদাহরণ ✍ অর্ডারিং বাস্তবায়ন অর্ডার: *R1* (টেপ) < *R2* (ডিস্ক) < *R3* (প্রিন্টার) < *R4* (প্লটার)। **(ক)** *P* চায় *R3* তারপর *R2*. *R2* ছোট, তাই প্রত্যাখ্যান; *P*-কে *R3* দিয়ে শেষ করে ছেড়ে দিতে হবে, তারপর আবার চাইতে হবে। **(খ)** *P* চায় *R1* তারপর *R4*. দুটোই ক্রমানুসারে বাড়ছে → **মঞ্জুর**। **(গ)** চক্র কি হতে পারে? প্রতিটি প্রসেস যে রিসোর্সের জন্য অপেক্ষা করছে তার চেয়ে ছোট নম্বরেরই ধরে আছে, তাই অপেক্ষার শৃঙ্খল ক্রমাগত **কমতে** থাকে → সার্কুলার ওয়েট অসম্ভব। **(ঘ)** *P*-এর কাছে *R4* আছে, সে *R4*-এর আরেকটি ইনস্ট্যান্স চায়। সমান, ক্রমানুসারে বড় নয় → **প্রত্যাখ্যান**: যে ধরনের ইনস্ট্যান্স ধরে আছে, তার আরেকটির জন্য কখনো অপেক্ষা করা যায় না।

মনে রাখার কৌশল সার্কুলার ওয়েট ভাঙাই **সাধারণত বলা উত্তর**, কারণ অর্ডারটি ডিজাইনার *a priori* নির্ধারণ করেন — অপারেটিং সিস্টেমের ভবিষ্যতের কোনো দাবি সম্পর্কে **কিছুই জানার দরকার নেই**। এভয়েড্যান্সে উল্টো প্রতিটি প্রসেসকে শুরুর আগেই সর্বোচ্চ দাবি ঘোষণা করতে হয়।

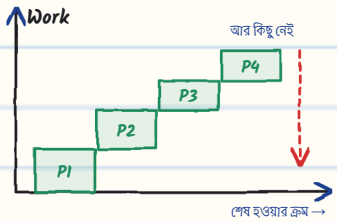
পরীক্ষার ফাঁদ *(i)* প্রতিরোধে **সব** রিসোর্স একসাথে চাইতে হয়; এভয়েড্যান্সে চাওয়া ব্যবহারযোগ্য রাখে কি না তা দেখা হয়। *(ii)* স্পুলার প্রিন্টারকে *কার্যত* শেয়ারযোগ্য করে, কিন্তু স্পুল ফাইল একচেটা। *(iii)* কনজিউমেবল রিসোর্স প্রিম্পট করা অসম্ভব।

বাস্তব ব্যবস্থা এটাই করে *Oracle*, *PostgreSQL* ও সতর্ক প্রতিটি কোডবেস লক অর্ডারিং আরোপ করে; দুই লক ছোঁয়া *Java* কোড সবসময় একই বৈশ্বিক ক্রমে নেয়। *A* তারপর *B* ও *B* তারপর *A* ধরা দুই থ্রেড সার্ভিস হ্যাং করবে — পাঠ্যবইয়ের চক্র, বাস্তবে।

৬ এভয়েড্যান্স: নিরাপদ অবস্থা Banker's ★

সংজ্ঞা **ব্যাংকার অ্যালগরিদম (Banker's Algorithm)**, ডেকস্ট্রা (Dijkstra, ১৯৬৫) — প্রসেস যা খুশি চাইতে পারে, কিন্তু **প্রতিটি অনুরোধ আগে যাচাই** করা হয় এবং তখনই মঞ্জুর হয় যখন ফলাফলের অবস্থায় সব প্রসেস শেষ হওয়ার সম্ভাবনা থাকে। দাম: প্রতিটি প্রসেসকে **আগেই** ঘোষণা করতে হয় যে সর্বোচ্চ কতটি ইনস্ট্যান্স সে কখনোই চাইতে পারে।

নিরাপদ অবস্থা (safe state) এমন একটি ক্রম $\langle P_1, \ldots, P_n \rangle$ — এটিকে বলে **সেফ সিকোয়েন্স (safe sequence)** — আছে যেখানে প্রতিটি প্রসেস তার **বাকি** চাহিদা তখনকার খালি পরিমাণ থেকে পূরণ করে শেষ হতে পারে, তারপর সব ফেরত দেয়। এমন ক্রম না থাকলে অবস্থাটি **অসুরক্ষিত (unsafe)**।



প্রতিটি ধাপ ওই মুহূর্তে খালি থাকা *Work* দিয়েই পূরণ হয়

ডেটা স্ট্রাকচার	মাপ	অর্থ
Available[j]	m	এখন ধরন j-এর খালি ইনস্ট্যান্স
Max[i][j]	$n \times m$	প্রসেস i-এর ঘোষিত সর্বোচ্চ দাবি
Allocation[i][j]	$n \times m$	প্রসেস i এখন যা ধরে আছে
Need[i][j]	$n \times m$	*Need = Max − Allocation*

পরীক্ষার ফাঁদ অসুরক্ষিত ≠ ডেডলক। অসুরক্ষিত অবস্থা মানে কেবল **ঘোষিত** সর্বোচ্চ দাবিগুলো আর কোনো ক্রমে মিটছে না; প্রকৃত চাহিদা ছোট হলে সব শেষ হতে পারে। **নিরাপদ অবস্থা নিশ্চিতভাবে ডেডলকমুক্ত**; অসুরক্ষিত শুধু একটি সতর্কবার্তা।

মনে রাখার কৌশল "*Need* মানে এখনো যা **চাইতে** হবে।" তাই ***Work*** শুধু ***Need***-এর সাথেই তুলনা হয়, আর প্রসেস শেষ হলে যা **ফেরত যায় তা *Allocation***। *Work* কখনো কমে না — শুধু বাড়ে, তাই লুপ কেন শেষ হয় তা স্পষ্ট।

চার শব্দের পার্থক্য

প্রতিরোধ: *আগেই* বিধান — "এটি কখনোই করবে না"

এভয়েড্যান্স: অনুরোধের সময় সিদ্ধান্ত — "হ্যাঁ, এতে নিরাপদ থাকে"

সনাক্তকরণ: আবার আবার চেক — "এখন কি এটা আছে?"

পুনরুদ্ধার: পরে পদক্ষেপ — "ভিকটিম বন্ধ বা প্রিম্পট করো"

সমাধানকৃত উদাহরণ ✍ নিরাপদ না অসুরক্ষিত একই রকমের চারটি টেপ ড্রাইভ, ***Available* = 2**। *P1* ধরে ২, আরও ০ দরকার; *P2* ধরে ০, দরকার ১; *P3* ধরে ০, দরকার ৪। *Need[P1] = 0 ≤ 2 ✓ → P1* শেষ হয়ে ২টি ফেরত দেয়: *Work = 4*। *Need[P2] = 1 ≤ 4 ✓*। *Need[P3] = 4 ≤ 4 ✓*। **সেফ সিকোয়েন্স ⟨P1, P2, P3⟩**। এখন *Available = 1* রাখলে: *P1* ও *P2* শেষ হয় (*Work = 3*), কিন্তু *P3*-এর ৪ > 3 → **অসুরক্ষিত**।

এভয়েড্যান্স শেখা হয়, কিন্তু কমই চালানো হয় প্রতিটি প্রসেসকে আগেই সর্বোচ্চ দাবি ঘোষণা করতে হয়, কিন্তু বাস্তব প্রোগ্রাম নিজের শীর্ষ মান সঠিকভাবে জানে না — তাই অপারেটিং সিস্টেম বেশি রিসোর্স আটকে রাখে ও ব্যবহার কমে যায়। নিরাপত্তা-পরীক্ষা সস্তা; **অনুমান করাটাই** দামি।

পরীক্ষার ফাঁদ ***Need[i]* ≤ *Work*-এর ≤ প্রতি উপাদানে** কাজ করে। *Need = (7, 5, 3), Work = (10, 4, 5)* হলে *5 > 4* বলে ব্যর্থ, যদিও মোট *15 < 19* দেখে ভালো লাগে। ভেক্টর কখনো যোগ করে মিলিয়ে নয়।

৭ ব্যাংকার অ্যালগরিদম, ধাপে ধাপে

লিখিত প্রশ্ন ★

অ্যালগরিদম ১ — সেফটি অ্যালগরিদম (অবস্থাটি কি নিরাপদ?)

১. **Work = Available**; সব *i*-এর জন্য **Finish[i] = false**।

২. এমন কোনো *i* খুঁজো যার **Finish[i] = false** এবং **Need[i] ≤ Work** (প্রতি উপাদানে)।

৩. তখন **Work = Work + Allocation[i]**; **Finish[i] = true**; ধাপ ২-এ ফিরে যাও।

৪. সব *Finish true* হলে → **সেফ**, আর সেই ক্রমটিই সেফ সিকোয়েন্স। কোনো *i false* থাকলে → **অসুরক্ষিত**।

অ্যালগরিদম ২ — রিকোয়েস্ট অ্যালগরিদম (অনুরোধ মঞ্জুর হবে?)

১. **Request[i] > Need[i]** হলে → **এরর**: ঘোষিত সর্বোচ্চ ছাড়িয়ে গেছে।

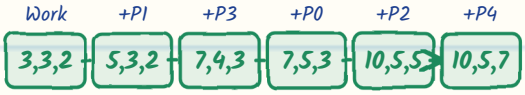
২. **Request[i] > Available** হলে → **অপেক্ষা**: এখন যথেষ্ট খালি নেই।

৩. **মনে মনে মঞ্জুর** করো: *Available −= Request; Allocation[i] += Request; Need[i] −= Request*।

৪. মনে-মানা অবস্থায় সেফটি অ্যালগরিদম চালাও। **সেফ → সত্যিই মঞ্জুর**, অসুরক্ষিত → **বাতিল** করে অপেক্ষা বলো।

প্রসেস	Allocation			Maximum			Need = Max − Alloc		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1
Available	**3**	**3**	**2**	মোট ইনস্ট্যান্স: A = 10, B = 5, C = 7					

ধাপ	প্রসেস	Need	Work আগে	Work পরে
1	P1	1 2 2	3 3 2 ✓	**5 3 2**
2	P3	0 1 1	5 3 2 ✓	**7 4 3**
3	P0	7 4 3	7 4 3 ✓	**7 5 3**
4	P2	6 0 0	7 5 3 ✓	**10 5 5**
5	P4	4 3 1	10 5 5 ✓	**10 5 7**

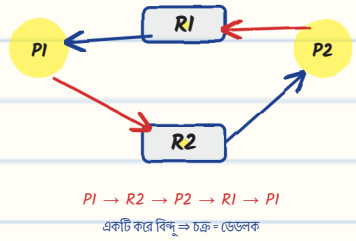


উপরের হাঁটার *Work* ভেক্টর — এটি কেবল বাড়ে

ফল ✍ এবার একটি অনুরোধ সব *Finish true* হয়েছে, তাই অবস্থাটি **সেফ** এবং সেফ সিকোয়েন্স ⟨P1, P3, P0, P2, P4⟩। যাচাই: শেষ *Work* (10, 5, 7) = *Available* + পুরো *Allocation*-এর যোগ, (3+7, 3+2, 2+5) = (10, 5, 7) ✓. এখন **Request[P1] = (1, 0, 2)**: ≤ *Need*[P1] = (1, 2, 2) ✓ এবং ≤ *Available* ✓। মনে-মানায়: *Available* = (2, 3, 0), *Allocation*[P1] = (3, 0, 2), *Need*[P1] = (0, 2, 0); হাঁটা P1 → (5, 3, 2), P3 → (7, 4, 3), P0 → (7, 5, 3), P2 → (10, 5, 5), P4 → (10, 5, 7) — পাঁচটিই শেষ → **মঞ্জুর**। কিন্তু **Request[P4] = (3, 3, 0)** দিলে *Available* = (0, 0, 2) ও *Need*[P4] = (1, 0, 1); কোনো প্রসেস মেলে না → **প্রত্যাখ্যান**।

৮ সনাক্তকরণ, পুনরুদ্ধার ও স্টারভেশন শেষ টপিক ★

সংজ্ঞা **সনাক্তকরণ (Detection)** ধরে নেয় যে অপারেটিং সিস্টেম ডেডলক হতে দেয়, তারপর নিয়মিত একটি পরীক্ষা চালায়। এটি *RAG*-ই রাখে, সঙ্গে ***Request[i][j]*** (প্রসেস এখনো কার জন্য অপেক্ষা করছে) ও ***Allocation[i][j]*** — ব্যাংকারের বহনই, কেবল প্রতি অনুরোধে নয়, একবার ব্যবহার করা হয়।



এক-ইনস্ট্যান্স সনাক্তকরণ: *RAG*-ই হলো অ্যালগরিদম

ক্ষেত্র	সনাক্তকরণ পদ্ধতি
প্রতিটি ধরনে **এক ইনস্ট্যান্স**	প্রতি বর্গে একটি বিন্দু দিয়ে *RAG* আঁকো; **যেকোনো চক্র = ডেডলক**; অতিরিক্ত ম্যাট্রিক্স লাগে না
একাধিক ইনস্ট্যান্স	পাতা ৫-এর **রিডিউশন** পরীক্ষা: খালি ইনস্ট্যান্স দিয়ে যে প্রসেসের রিকোয়েস্ট মেটে তাকে শেষ করো ও তার *Allocation* ফেরত যোগ করো; যা বাকি থাকে তাই ডেডলক
খরচ	$O(n^2)$ বা তার বেশি — তাই বাস্তব সিস্টেম এটি করে না

পুনরুদ্ধার ১ — প্রসেস টার্মিনেশন ডেডলক করা **সব** প্রসেস বন্ধ করো, অথবা **একটি করে বেছে নেওয়া ক্রমে** বন্ধ করো যতক্ষণ না চক্র ভাঙে; বন্ধ করা প্রসেস যা ধরে ছিল সব ছেড়ে দেয়। সাধারণ ক্রম, কম ক্ষতি হবে এমন: **এখন পর্যন্ত সবচেয়ে কম *CPU* ব্যবহার → সবচেয়ে কম রিসোর্স ধরা → সবচেয়ে কম গুরুত্বপূর্ণ রিসোর্স → বাকি সবচেয়ে দীর্ঘ *CPU***।

পুনরুদ্ধার ২ — রিসোর্স প্রিম্পশন একটি ভিকটিমের কাছ থেকে **রিসোর্স কেড়ে নিয়ে** ব্লক করা প্রসেসকে দাও। ভিকটিমকে ফেরত আনা যায় কেবল রোলব্যাক *(rollback)* করে — আগের নিরাপদ অবস্থায় ফেরানো বা চেকপয়েন্ট থেকে শুরু করে — কারণ সে নেওয়া রিসোর্সটির **আংশিক ব্যবহার** করে ফেলেছে। **স্টারভেশন ঠেকাতে:** রোলব্যাকের সংখ্যা গুনে একই প্রসেসকে বারবার প্রিম্পট করা এড়ান।

বিষয়	ডেডলক	স্টারভেশন
কী হয়	প্রসেসের একটি **সেট** পরস্পরের ওপর অপেক্ষা করছে; সেটের কেউ এগোতে পারে না	একটি প্রসেস যা চায় তা **অনির্দিষ্টকাল** পায় না
কে ক্ষতিগ্রস্ত	**চক্রের প্রতিটি প্রসেস**	শুধু অসমী প্রসেস; বাকিরা এগোতে থাকে
আরোগ্য	টার্মিনেশন, বা প্রিম্পশন + রোলব্যাক	**এজিং *(aging)***, ন্যায্য সারি

মনে রাখার কৌশল **"ডেডলক পুরো জগৎ থামায়, স্টারভেশন একজনকে বেছে নেয়।"** ডেডলক = একটি **সেট** একসাথে আটকে থাকা; স্টারভেশন = একটি **একক** প্রসেস বঞ্চিত হওয়া, অথচ অবস্থা অবিরাম বদলায়। আর এজিং কখনো ডেডলক ঠিকায় না — কিছুই এগোচ্ছে না, তাই অগ্রাধিকার পুনর্মূল্যায়নও হয় না।

বাস্তবের মূল কথা বাস্তব অপারেটিং সিস্টেম ও ডেটাবেস **সনাক্তকরণ না করে নকশা করেই ডেডলক এড়িয়ে চলে: অর্ডারযুক্ত লকিং** (লকের একটি বৈশ্বিক টোটাল অর্ডার) এবং **হোল্ড-অ্যান্ড-ওয়েট নিষিদ্ধ** (একসাথে একটি লক, পরেরটি চাওয়ার আগে আগেরটি ছেড়ে দেওয়া)। চক্রের সম্ভাবনা থাকলেও তারা **টাইমআউট** বসায় — লক টাইমার, ডেডলাইনসহ `tryLock`, স্ট্যাক ডাম্প করে বাতিল করা একটি ওয়াচডগ।

রাতের-পরীক্ষার পাতা

তথ্য-পত্র

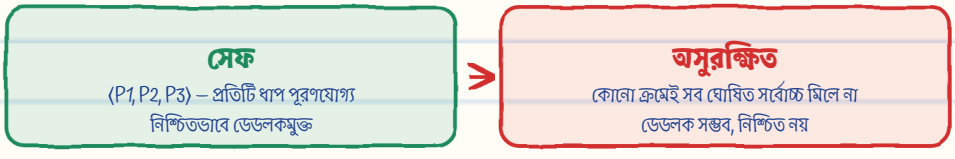
কফম্যান (১৯৭১): **M** মিউচুয়াল এক্সক্লুশন	**H** হোল্ড অ্যান্ড ওয়েট
N নো প্রিম্পশন	**C** সার্কুলার ওয়েট
ক্লেইম এজ **R** → **P** (বরাদ্দ)	রিকোয়েস্ট এজ **P** → **R**
১ ইনস্ট্যান্স ⇒ চক্র = ডেডলক	অনেক ⇒ রিডিউশন চালাও
Need = Max − Allocation	**Work = Available**
Work += Allocation[i]	সব *Finish true* ⇒ **সেফ**
Request > Need ⇒ এরর	*Request > Available* ⇒ অপেক্ষা
অসুরক্ষিত ≠ ডেডলক	অর্ডার ঠিক হয় ***a priori***
রিইউজেবল বনাম কনজিউমেবল	টার্মিনেট: কম *CPU* ব্যবহার
প্রিম্পট ⇒ **রোলব্যাক**	এজিং ⇒ শুধু স্টারভেশন

➜ MCQ-র জন্য সেরা ১২টি এক-লাইনের কথা

1. **চারটি** শর্তই লাগে
2. শুধু **অন্তত একটি** রিসোর্স একচেটা
3. ডেডলক ⇒ চক্র; চক্র ⇒ ডেডলক **কেবল** এক-ইনস্ট্যান্সে
4. ক্লেইম **R** → **P**, রিকোয়েস্ট **P** → **R**
5. বৃত্ত = প্রসেস, বর্গ = রিসোর্স, বিন্দু = ইনস্ট্যান্স
6. প্রতিরোধ: **সার্কুলার ওয়েট ভাঙো**
7. *Banker's* = **এভয়েড্যান্স** (*Dijkstra*)
8. ***Need = Max − Allocation***
9. *Work = Work + Allocation[i]*
10. অসুরক্ষিত শুধু একটি **সতর্কবার্তা**
11. পুনরুদ্ধার: **টারমিনেট** বা **প্রিম্পট + রোলব্যাক**
12. প্রিন্টারের ক্ষেত্রে প্রিম্পশন **অসম্ভব**

সবচেয়ে বড় ফাঁদ এক-ইনস্ট্যান্সের শর্ত ছাড়া "চক্র = ডেডলক" · *Need ≠ Max − Available* · ব্যবহৃত ডিভাইসের জন্য অপেক্ষা **ডেডলক নয়** · "অন্তত একটি" রিসোর্স, সব নয় · অসুরক্ষিত ≠ ডেডলক · এজিং স্টারভেশন ঠিকায়, ডেডলক কখনো নয় · যে প্রসেস তার চাওয়া সব ইনস্ট্যান্স ধরে আছে, তার বহির্গত রিকোয়েস্ট এজ নেই এবং সে **ডেডলকে নেই**।

সব কৌশল একসাথে "ডেডলকের জন্য অন্তত দুজন, আর চারটিই লাগে" · "চার দরজার ঘর — একটিও খুললে মুক্ত" · *MHNC* · "এক বিন্দু, এক উত্তর" · "*R* ফেরত হয়, *C* ক্ষয় হয়" · "তির ধারক থেকে বেরিয়ে চাওয়ার দিকে যায়" · "অর্ডার আগেই ঠিক (*a priori*)" · "*Need* মানে এখনো যা চাইতে হবে; *Work* কেবল বাড়ে" · "ডেডলক পুরো জগৎ থামায়, স্টারভেশন একজনকে বেছে নেয়"।



Banker কেবল বাঁ পাশের অবস্থা মঞ্জুর করে

এভয়েড্যান্সের পুরো ধারণা দুটি বাক্সে

✓ অনুশীলনী MCQ (পরীক্ষার ধরনে, ১০ নম্বর)

আগে চেষ্টা করো, তারপর দেখো

১. ডেডলক কখনই হতে পারে:
ক) চারটি শর্তই থাকলে খ) যেকোনো তিনটি থাকলে
গ) শুধু সার্কুলার ওয়েট থাকলে ঘ) মিউচুয়াল এক্সক্লুশন না থাকলে

২. প্রতিটি রিসোর্সের একটি ইনস্ট্যান্স:
ক) হয়তো ডেডলক হয় খ) নিশ্চিতভাবে ডেডলক হয়
গ) নিরাপদ অবস্থা ঘ) শুধু স্টারভেশন

৩. একটি ক্লেইম (অ্যালোকেশন) এজ:
ক) রিসোর্স → ধারক খ) প্রসেস → চাওয়া ধরন
গ) প্রসেস ↔ প্রসেস ঘ) ধরন ↔ ধরন

৪. সাধারণত ব্যবহৃত প্রতিরোধ:
ক) প্রিম্পশনের অনুমতি খ) সব একসাথে চাওয়া
গ) একটি টোটাল অর্ডার ঘ) সব শেয়ারযোগ্য করা

৫. *Banker's* অ্যালগরিদম কীসের জন্য:
ক) সনাক্তকরণ খ) প্রতিরোধ
গ) এভয়েড্যান্স ঘ) পুনরুদ্ধার

৬. *Need[i][j]* সমান:
ক) *Max + Alloc* খ) *Avail − Alloc*
গ) *Max − Avail* ঘ) *Max − Alloc*

৭. *Work Available* থেকে শুরু, তারপর:
ক) *+ Max[i]* খ) *− Alloc[i]*
গ) *+ Need[i]* ঘ) *+ Alloc[i]*

৮. প্রিম্পশনের পর ভিকটিম:
ক) রোলব্যাক হয় খ) যেমন ছিল চলে
গ) স্থায়ী বন্ধ হয় ঘ) নতুন লক পায়

৯. চার শর্তের **কোনটি নয়**:
ক) হোল্ড অ্যান্ড ওয়েট খ) এজিং
গ) সার্কুলার ওয়েট ঘ) মিউচুয়াল এক্সক্লুশন

১০. টোটাল অর্ডারের জন্য আগে কী লাগে:
ক) প্রসেসের সর্বোচ্চ দাবি খ) কিছুই নয়, *a priori*
গ) *CPU* বাস্টের আকার ঘ) ডিস্কের গঠন

উত্তরমালা → ১-ক · ২-খ · ৩-ক · ৪-গ · ৫-গ · ৬-ঘ · ৭-ঘ · ৮-ক · ৯-খ · ১০-ঘ

লিখিত প্রশ্ন কফম্যানের চার শর্ত লেখো ও প্রতিরোধ কীভাবে প্রতিটি অসম্ভব করে। দুই ডেডলক করা প্রসেসের *RAG* আঁকো এবং প্রতি ধরনে একটি ইনস্ট্যান্স হলে চক্র কী প্রমাণ করে তা বলো। টেবিলে দেওয়া *P0–P4*-এর *Allocation* ও *Maximum* এবং *Available = (3, 3, 2)* দিয়ে সেফ সিকোয়েন্স বের করো এবং *Request[P1] = (1, 0, 2)* মঞ্জুর হবে কি না বিচার করো। ১০ নম্বর

১ **শর্ত** — *(i)* **মিউচুয়াল এক্সক্লুশন**: অন্তত একটি রিসোর্স অবাগত্যোগ্য; *(ii)* **হোল্ড অ্যান্ড ওয়েট**: একটি ধরে রেখে আরও কার জন্য অপেক্ষা; *(iii)* **নো প্রিম্পশন**: কেবল স্বেচ্ছায় ছাড়া হয়; *(iv)* **সার্কুলার ওয়েট**: বদ্ধ শৃঙ্খল *P0 → … → Pn → P0*। প্রতিরোধ: শেয়ারযোগ্য রিসোর্স বা স্পুলার; সব একসাথে চাওয়া বা ছেড়ে দিয়ে চাওয়া; *CPU*/মেমোরি/ফাইল প্রিম্পশন (প্রিন্টার কখনো নয়); টোটাল অর্ডার। ৩

২ **গ্রাফ** — *P1, P2* বৃত্ত; *R1, R2* একটি করে বিন্দুসহ বর্গ; ক্লেইম এজ *R1 → P1* ও *R2 → P2*, রিকোয়েস্ট এজ *P1 → R2* ও *P2 → R1*। একটি ইনস্ট্যান্স প্রতি ধরনে হলে চক্র **প্রয়োজনীয় ও পর্যাপ্ত**। ৩

৩ **হাটা** — *Need = 7 4 3 / 1 2 2 / 6 0 0 / 0 1 1 / 4 3 1*. *Work (3, 3, 2) → P1 (5, 3, 2) → P3 (7, 4, 3) → P0 (7, 5, 3) → P2 (10, 5, 5) → P4 (10, 5, 7)* ⇒ **সেফ**, ⟨*P1, P3, P0, P2, P4*⟩। ৩

৪ **অনুরোধ** — *(1, 0, 2) ≤ Need[P1] = (1, 2, 2)* ✓ এবং ≤ *Available* ✓। মনে-মানায় *Available = (2, 3, 0)*, *Need[P1] = (0, 2, 0)*; সেফটি হাঁটায় পাঁচটিই শেষ ⇒ **মঞ্জুর**। ১

→ পরীক্ষার আগে ✓

✓ মুখস্থভাবে চার শর্ত বলতে পারি
✓ *RAG* আঁকতে পারি, ক্লেইম বনাম রিকোয়েস্ট বলতে পারি
✓ ৫ × ৩ টেবিলে সেফটি অ্যালগরিদম চালাতে পারি
✓ স্টারভেশন আর ডেডলক এক লাইনে পার্থক্য করতে পারি